Study Library Management System (SLMS)

A modern, production-grade library seat reservation and administration platform designed for seamless library operations, real-time seat allocations, and role-based workflows.



Table of Contents

- Overview
- System Architecture & Roles
- Administrator Portal
- Staff Portal
- Student Portal
- Tech Stack
- Quick Start & Setup Guide
- Prerequisites
- 1. Clone / Navigate to Repository
- 2. Environment Setup
  - Windows (PowerShell)
  - Windows (Command Prompt)
  - macOS & Linux (Bash / Zsh)
- 3. Configure Environment Variables
- 4. Run the Application
- Default Seed Credentials
- Project Structure
- API Reference
- Running Automated Tests
- Production Deployment
- License

Overview

The **Study Library Management System (SLMS)** provides an end-to-end management ecosystem for study libraries, reading rooms, and co-working study spaces. It eliminates manual physical registers by offering real-time seat reservation, automated attendance & check-in verification, membership subscriptions, complaint ticketing, offline payment logging, and actionable administrative analytics.

System Architecture & Roles



Administrator Portal

- **Staff Accounts**: Create, edit, activate, or deactivate library staff accounts.
- **Membership Approvals**: Review pending student registrations with instant Approve, Reject, or Suspend actions.
- **Operating Hours**: Configure weekly schedules (Monday–Sunday) and holiday closure dates.
- **Capacity & Limits**: Adjust maximum booking hours per day, concurrent seat allocations, waitlist rules, and zone definitions (Quiet, Discussion, Computer Lab).
- **Plans & Pricing**: Create and manage flexible membership tiers (Weekly, Monthly, Yearly, Custom).
- **Analytics & Reports**: Visual charts for occupancy rates, peak study hours, revenue streams, and membership distribution.
- **Broadcasts**: Publish library announcements targeted to All, Staff, or Students.

Staff Portal

- **Check-In & Attendance**: Verify student reservations via check-in codes or manual roster lookups.
- **Walk-in Allocations**: Instantly allocate available seats to same-day walk-in students.
- **Extension Requests**: Review and approve student requests for extra study hours.
- **Fee Management**: Log cash/offline payment receipts against student accounts.
- **Student Onboarding**: Register new student profiles and assign membership plans.
- **Complaint Helpdesk**: Review student-reported issues, add resolution notes, and update ticket statuses.

Student Portal

- **Interactive Space Booking**: Filter by date, time slot (Morning, Afternoon, Evening, Full Day), and seat zones.
- **Live Seat Availability**: View real-time seat availability matrix.
- **My Bookings**: View active reservations, copy check-in codes, request time extensions, cancel reservations, or perform self check-out.
- **Session History**: Track total logged study hours and past attendance records.
- **Fee Receipts**: Review active membership validity and payment transaction history.
- **Help & Complaints**: Submit tickets categorized by AC/Temperature, Noise, Wi-Fi, Seat Damage, or Other.

Tech Stack

- **Backend**: Python 3.10+, Flask 3.1, Flask-RESTful, Flask-JWT-Extended, Werkzeug Security
- **Database & ORM**: SQLAlchemy 2.0, SQLite (default) / PostgreSQL / MySQL ready
- **Frontend**: Vue 3 (Reactive Composition Engine), Vanilla CSS with custom glassmorphism design tokens
- **WSGI & Dev Servers**: Werkzeug, Waitress WSGI
- **Testing & QA**: Pytest, Playwright Browser Automation

Quick Start & Setup Guide

Prerequisites

- **Python 3.10 or higher** installed (Download Python)
- **Git** installed (Download Git)

---

1. Clone / Navigate to Repository

```
git clone https://github.com/GyaanFlow/MAY2026-Team-051.git
cd MAY2026-Team-051
```

---

2. Environment Setup

Windows (PowerShell)

```
# Create virtual environment
python -m venv venv

# Activate virtual environment
.\venv\Scripts\Activate.ps1

# Upgrade pip and install dependencies
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Windows (Command Prompt)

```
:: Create virtual environment
python -m venv venv

:: Activate virtual environment
venv\Scripts\activate.bat

:: Upgrade pip and install dependencies
python -m pip install --upgrade pip
pip install -r requirements.txt
```

macOS & Linux (Bash / Zsh)

```
# Create virtual environment
python3 -m venv venv

# Activate virtual environment
source venv/bin/activate

# Upgrade pip and install dependencies
python3 -m pip install --upgrade pip
pip install -r requirements.txt
```

---

3. Configure Environment Variables

Create your local `.env` configuration file from the template:

Windows (PowerShell):

```
Copy-Item .env.example .env
```

macOS / Linux:

```
cp .env.example .env
```

Default configuration in `.env`:

```
FLASK_ENV=development
SECRET_KEY=dev-secret-key-change-in-production
JWT_SECRET_KEY=jwt-secret-key-change-in-production
DATABASE_URL=sqlite:///instance/app.db
JWT_ACCESS_TOKEN_EXPIRES_HOURS=8
CORS_ORIGINS=*
HOST=127.0.0.1
PORT=5000
```

---

4. Run the Application

Execute the application runner (the database and seed data will initialize automatically on first startup):

```
python app.py
```

The application will be live at: **`http://127.0.0.1:5000`**

Default Seed Credentials

When starting the application for the first time, default seed data is automatically populated:

Role	Username	Password	Access URL
** Admin**	`admin`	`test-password`	`http://127.0.0.1:5000/` $\rightarrow$ `/admin/dashboard`

> **Tip**
>
> - **Staff Accounts**: Log in as `admin`, navigate to **Staff Account Management**, and click **Create Staff Account**.
> - **Student Accounts**: Log in as `staff`, navigate to **Create Student Account**, and register the student. Then log back in as `admin` under **Student Membership Management** to approve their membership.

Project Structure

```
├── app.py                      # Application factory, route registrations & see
├── config.py                   # Environment settings, JWT & database configura
├── models.py                   # SQLAlchemy database models (Users, Students, S
├── utils.py                    # Role-based decorators and helper utilities
├── requirements.txt            # Production & development dependencies
├── .env.example                # Environment variables template
├── .gitignore                  # Git ignore rules for Python, IDEs, and environ
│
├── resources/                  # RESTful API controllers
│   ├── auth.py                 # JWT authentication & login controller
│   ├── profile.py              # User profile updates & password resets
│   ├── admin_memberships.py    # Admin membership approvals & suspensions
│   ├── staff.py                # Admin staff account management
│   ├── staff_students.py       # Staff student registration & ID verification
│   ├── membership.py           # Membership plan catalog
│   ├── seats.py                # Seat availability, zoning, and inventory
│   ├── bookings.py             # Seat reservations, checkouts, and walk-ins
│   ├── checkin.py              # Check-in code verification
│   ├── extension_requests.py   # Student study time extension requests
│   ├── no_show.py              # No-show candidate evaluation & seat release
│   ├── payments.py             # Fee payment history & offline receipt recordin
│   ├── complaints.py           # Helpdesk & complaint resolution
│   ├── announcements.py        # Global broadcasts & notices
│   ├── reports.py              # Occupancy, revenue, and peak-hours analytics
│   └── settings.py             # Library operating hours & capacity limits
│
├── templates/                  # Jinja2 / HTML templates
│   ├── login.html              # Unified authentication portal
│   ├── admin_dashboard.html    # Administrator workspace
│   ├── staff_dashboard.html    # Library staff operations desk
│   └── dashboard.html          # Student portal & booking interface
│
└── static/                     # Frontend assets
    ├── css/
    │   ├── style.css           # Design system, auth styling & utility classes
    │   └── dashboard.css       # Dashboard layout, sidebar, stat cards & forms
    └── js/
        ├── api.js              # Fetch wrapper with JWT headers & error handlin
        ├── auth-guard.js       # Client-side route protection & session guard
        ├── auth.js             # Login form controller
        ├── admin_dashboard.js  # Admin SPA controller
        ├── staff_dashboard.js  # Staff SPA controller
        ├── dashboard.js        # Student SPA controller
        ├── library-data.js     # Time slot helpers & mock data constants
        └── toast.js            # Toast notification engine
```

API Reference

Authentication & Profile

Method	Endpoint	Access	Description
`POST`	`/api/auth/login`	Public	Authenticate user and return JWT access token
`GET`	`/api/profile`	Authenticated	Retrieve current user profile
`PUT`	`/api/profile`	Authenticated	Update user personal information
`POST`	`/api/auth/reset-password`	Authenticated	Change user password

Administrator Endpoints

Method	Endpoint	Access	Description
`GET`, `POST`	`/api/admin/staff`	Admin	List staff members / Create staff account
`PUT`	`/api/admin/staff/<id>`	Admin	Update staff account details
`POST`	`/api/admin/staff/<id>/deactivate`	Admin	Deactivate staff account
`POST`	`/api/admin/staff/<id>/activate`	Admin	Activate staff account
`GET`	`/api/admin/memberships/pending`	Admin	List pending membership approval requests
`POST`	`/api/admin/memberships/<id>/approve`	Admin	Approve student membership
`POST`	`/api/admin/memberships/<id>/reject`	Admin	Reject student membership

Method	Endpoint	Access	Description
`GET`, `POST`	`/api/admin/membership-plans`	Admin	List / Create membership plans
`GET`, `PUT`	`/api/admin/settings`	Admin	Retrieve / Update operating hours & limits
`GET`	`/api/admin/reports/occupancy`	Admin	Retrieve occupancy percentage analytics
`GET`	`/api/admin/reports/revenue`	Admin	Retrieve revenue report breakdown
`GET`	`/api/admin/reports/peak-hours`	Admin	Retrieve peak usage hours data
`GET`, `POST`	`/api/admin/announcements`	Admin	View / Publish library announcements

Staff Operations Endpoints

Method	Endpoint	Access	Description
`GET`	`/api/staff/bookings/today`	Staff	Retrieve today's booking roster
`POST`	`/api/staff/checkin`	Staff	Verify check-in code and check student in
`POST`	`/api/staff/bookings/walk-in`	Staff	Allocate seat to walk-in student
`POST`	`/api/staff/students/register`	Staff	Register new student profile
`GET`	`/api/staff/extension-requests`	Staff	List student extension requests

Method	Endpoint	Access	Description
`POST`	`/api/staff/extension-requests/<id>/approve`	Staff	Approve study extension request
`POST`	`/api/staff/payments`	Staff	Record manual / offline fee payment
`GET`, `PUT`	`/api/staff/complaints/<id>`	Staff	List complaints / Update complaint resolution

Student Workspace Endpoints

Method	Endpoint	Access	Description
`GET`	`/api/seats/availability`	Authenticated	Live seat occupancy and availability status
`GET`, `POST`	`/api/bookings`	Student	List active bookings / Create new seat reservation
`POST`	`/api/bookings/<id>/cancel`	Student	Cancel upcoming seat reservation
`POST`	`/api/bookings/<id>/checkout`	Student	Self checkout from current seat
`POST`	`/api/students/me/bookings/<id>/extension-request`	Student	Request booking extension
`GET`	`/api/students/me/payments`	Student	Retrieve student fee & transaction history

Method	Endpoint	Access	Description
`POST`	`/api/students/me/complaints`	Student	Submit complaint or issue ticket

Running Automated Tests

The repository includes a comprehensive test suite covering backend logic, JWT security, and API endpoints.

```
# Run all unit and integration tests
pytest

# Run tests with verbose output
pytest -v

# Run with coverage summary
pytest --cov=.
```

Production Deployment

For production environments, run behind a production WSGI server such as **Waitress** (Windows / Cross-platform) or **Gunicorn** (Linux / macOS):

Using Waitress (Windows / Linux / macOS)

```
waitress-serve --listen=0.0.0.0:5000 app:create_app
```

Using Gunicorn (Linux / macOS)

```
gunicorn -w 4 -b 0.0.0.0:5000 "app:create_app()"
```

License

This project is licensed under the MIT License — see the repository for details.